DATA SCIENCE

NumPy

The vectorized ndarray that powers the entire PyData stack — master arrays, broadcasting, and why loops are the enemy.

numpy · ndarray · vectorization · broadcasting · linear-algebra · data-science · python

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What NumPy Is and Why It Matters
2. Setup and Getting Started
3. The ndarray: Shape, dtype, and Strides
4. Creating Arrays
5. Vectorization: Why Loops Are the Enemy
6. Broadcasting
7. Indexing and Slicing
8. Reshaping and Combining
9. Aggregation and Axis Semantics
10. Linear Algebra, Random, and More
11. Worked Mini-Example: K-Means Assignment Step
12. Cheat-Sheet Reference
13. Best Practices and Common Pitfalls
14. Interview Questions & Answers
15. Further Learning

What NumPy Is and Why It Matters

NumPy (Numerical Python) is the foundational library for numerical computing in Python. It provides the `ndarray` — a dense, homogeneous, N-dimensional array — plus a vast library of fast, vectorized operations written in C. Nearly every scientific Python library (pandas, scikit-learn, PyTorch, TensorFlow, SciPy, Matplotlib) is built on top of it.

A Python `list` is a flexible array of pointers to boxed objects scattered across memory. A NumPy array is a single contiguous block of raw, fixed-width values with one shared type. That difference is

the entire story: contiguous typed memory lets the CPU stream data through cache and SIMD units, and lets NumPy run tight C loops instead of the interpreter's per-element overhead.

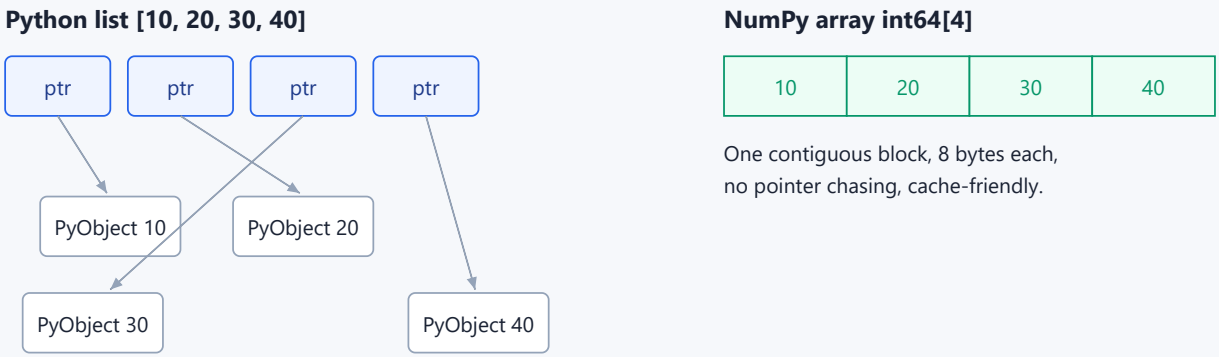


Fig 1. A list stores pointers to scattered boxed objects; an ndarray stores raw values packed contiguously.

✦ **Mental model**

Think of an ndarray as a *typed C buffer* plus a small Python header describing its `shape`, `dtype`, and `strides`. Operations don't loop in Python — they dispatch to compiled routines that walk the buffer.

Setup and Getting Started

NumPy ships with every scientific Python distribution and installs cleanly via pip or conda. The near-universal import alias is `np`.

BASH

```bash
# Install (pick one)
pip install numpy
conda install numpy

# Verify
python -c "import numpy as np; print(np.__version__)"
# e.g. 2.3.x  (NumPy 2.x is the current major line in 2026)
```

PYTHON

```python
import numpy as np

a = np.array([1, 2, 3, 4])          # from a Python list
print(a)                             # [1 2 3 4]
print(a.dtype, a.shape, a.ndim)      # int64 (4,) 1
print(a.nbytes)                      # 32  (4 elements * 8 bytes)
```

> **NumPy 2.x**
> NumPy 2.0 (2024) was the first ABI-breaking release in years. Default integer on Windows is now 64-bit, `np.float_` / `np.int0` and other aliases were removed, and `np.copy` defaults changed. Code pinned to 1.x may need small fixes — see the official migration guide.

The ndarray: Shape, dtype, and Strides

Three attributes fully describe how an array is laid out and interpreted. Understanding them demystifies views, reshaping, and broadcasting.

SHAPE

A tuple of dimension sizes. `(3, 4)` is 3 rows by 4 columns. `ndim` is `len(shape)`; `size` is the product.

DTYPE

The element type: `int64`, `float64`, `bool`, `complex128`, etc. Every element shares it; this is what makes the buffer homogeneous.

STRIDES

Bytes to step in memory to advance one index along each axis. Reshaping and slicing often just rewrite strides — no data copy.

PYTHON

```python
m = np.arange(12).reshape(3, 4)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]
print(m.shape)    # (3, 4)
print(m.strides)  # (32, 8)  → 32 bytes to next row, 8 to next column
print(m.dtype)    # int64
```

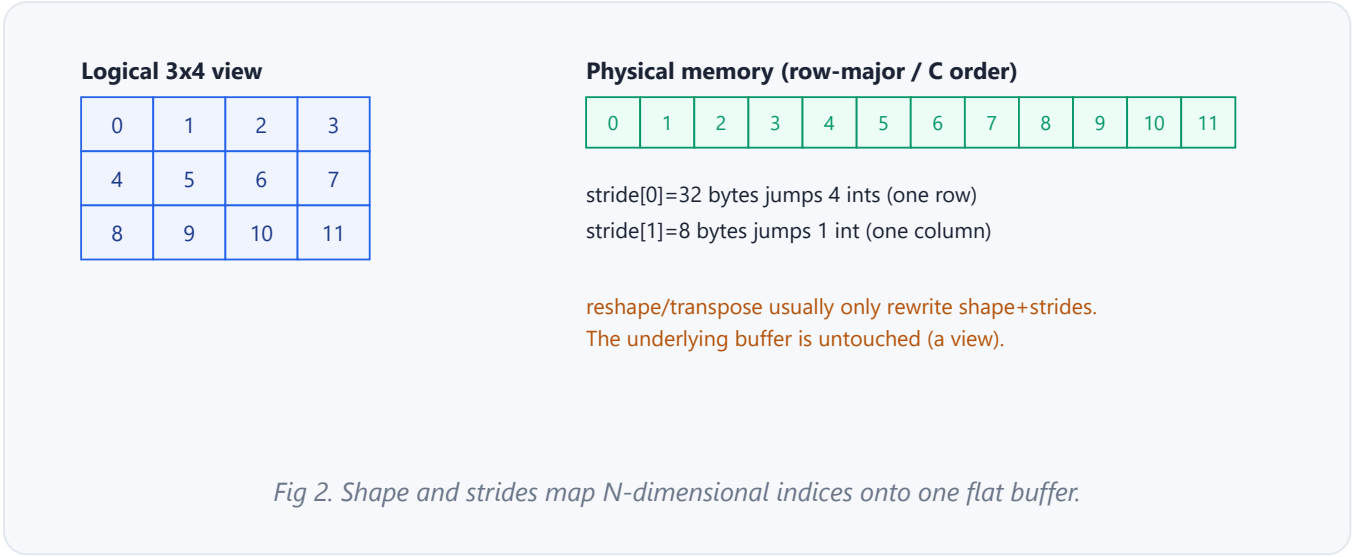


Fig 2. Shape and strides map N-dimensional indices onto one flat buffer.

dtypes

Choosing the right dtype controls both memory and numeric behavior. Integer types wrap on overflow; floats follow IEEE-754.

dtype	Bytes	Range / notes
`int8 / int16 / int32 / int64`	1 / 2 / 4 / 8	Signed, wraps silently on overflow
`uint8`	1	0–255; images, masks
`float32`	4	~7 significant digits; GPU / ML default
`float64`	8	~15–16 digits; NumPy default for floats
`complex128`	16	Two float64 (real + imag)
`bool`	1	Boolean masks
`<U10`	varies	Fixed-width Unicode strings (avoid for big data)

PYTHON

```
x = np.array([1, 2, 3], dtype=np.float32)
y = x.astype(np.int32)              # explicit cast → [1 2 3]

# Overflow wraps silently — a classic bug:
np.array([255], dtype=np.uint8) + np.array([1], dtype=np.uint8)  # array([0], uint8)

# Promotion rules: mixing types up-casts to a common dtype
(np.array([1, 2], dtype=np.int32) + 0.5).dtype   # float64
```

Creating Arrays

You rarely build big arrays from Python lists. Use the constructors — they allocate the buffer directly.

Call	Result
`np.zeros((2,3))`	2x3 of 0.0
`np.ones(5, dtype=int)`	[1 1 1 1 1]
`np.full((2,2), 7)`	2x2 of 7
`np.arange(0, 10, 2)`	[0 2 4 6 8]
`np.linspace(0, 1, 5)`	[0. 0.25 0.5 0.75 1.]
`np.eye(3)`	3x3 identity
`np.empty((2,2))`	Uninitialized (garbage) — fastest
`np.zeros_like(a)`	Zeros matching a's shape+dtype

PYTHON

```
grid = np.linspace(0, 2*np.pi, 100)   # 100 evenly spaced points
mat  = np.eye(4)                       # identity matrix
rng  = np.arange(12).reshape(3, 4)     # 0..11 shaped 3x4
blank = np.full((3, 3), np.nan)        # NaN-filled float grid
```

Vectorization: Why Loops Are the Enemy

Vectorization means expressing a computation as whole-array operations so the heavy lifting happens in compiled C, not the Python interpreter. This is the single most important NumPy skill.

PYTHON

```python
# SLOW: Python-level loop, ~millions of interpreter steps
def slow_sqdist(a, b):
    out = np.empty(len(a))
    for i in range(len(a)):
        out[i] = (a[i] - b[i]) ** 2
    return out

# FAST: one vectorized expression, runs entirely in C
def fast_sqdist(a, b):
    return (a - b) ** 2

a = np.random.rand(1_000_000)
b = np.random.rand(1_000_000)
# fast_sqdist is typically 50-200x faster and far more readable.
```

✦ **The rule of thumb**

If you write a Python `for` loop over array elements, pause — there is almost always a vectorized form using arithmetic, a ufunc, broadcasting, or boolean masking. Loops in Python pay interpreter overhead *per element*; vectorized ops pay it *once*.

Why is it faster? Three reasons compound:

NO INTERPRETER LOOP

One C loop replaces millions of bytecode dispatches.

CACHE + SIMD

Contiguous data streams through CPU cache; compilers emit SIMD (AVX) instructions processing multiple values per cycle.

NO BOXING

Raw `int64` / `float64` math, not boxed `PyObject` allocations.

Universal functions (ufuncs)

A ufunc operates element-wise and supports broadcasting, type promotion, and an `out=` parameter. Examples: `np.sin`, `np.exp`, `np.sqrt`, `np.maximum`, `+`, `*`.

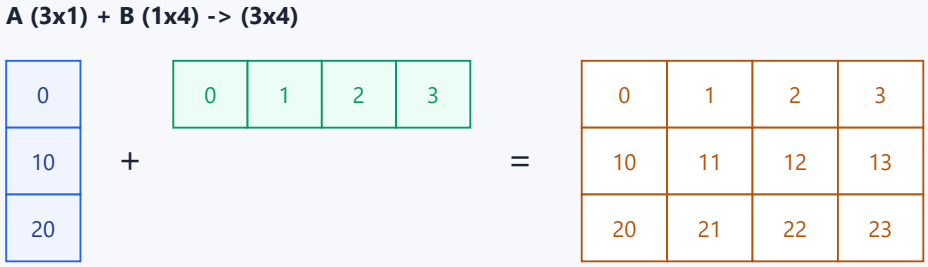
PYTHON

```python
x = np.linspace(0, np.pi, 5)
np.sin(x)                       # element-wise sine
np.exp(x) + np.sqrt(x)          # chained ufuncs, each one C loop
np.maximum(x, 1.0)              # element-wise max against a scalar
np.add(x, x, out=x)             # write result in place, no new allocation
```

Broadcasting

Broadcasting is the set of rules that let NumPy combine arrays of different shapes without copying data. Smaller arrays are "stretched" virtually across the larger one.

> **The rules**
> Compare shapes right-to-left. Two dimensions are compatible when they are **equal**, or **one of them is 1**. A missing leading dimension is treated as 1. The size-1 (or missing) axis is virtually repeated.



Shape alignment (right to left):
A: (3, 1) -> result (3, 4): each size-1 axis is stretched.
B: (1, 4)

Fig 3. Broadcasting stretches size-1 axes virtually — no data is actually copied.

PYTHON

```python
# Add a row vector to every row of a matrix
M = np.zeros((3, 4))
row = np.array([1, 2, 3, 4])
M + row                        # row broadcast over all 3 rows → (3,4)

# Outer product via broadcasting
col = np.array([0, 10, 20]).reshape(3, 1)   # (3,1)
rowv = np.array([0, 1, 2, 3])                # (4,)
col + rowv                                   # (3,4) grid

# Standardize columns (subtract mean, divide by std) — pure broadcasting
data = np.random.rand(100, 5)
z = (data - data.mean(axis=0)) / data.std(axis=0)   # (100,5)-(5,) ok
```

> ⚠ **Shape mismatch**
>
> `np.ones((3,4)) + np.ones((2,4))` raises `ValueError: operands could not be broadcast together`. The trailing axes (4 vs 4) match, but 3 vs 2 are both >1 and unequal. Use `reshape` or `np.newaxis` to add a size-1 axis deliberately.

Indexing and Slicing

NumPy offers three indexing styles. **Basic slicing returns a view** (shares memory); **fancy and boolean indexing return copies**.

PYTHON

```python
a = np.arange(10)              # [0 1 2 3 4 5 6 7 8 9]
a[2:5]                         # [2 3 4]      (view)
a[::-1]                        # reversed     (view)
a[1::2]                        # [1 3 5 7 9]  (view, step 2)

M = np.arange(12).reshape(3, 4)
M[1, 2]                        # 6            (scalar)
M[0]                           # [0 1 2 3]    (first row, view)
M[:, 1]                        # [1 5 9]      (second column, view)
M[0:2, 1:3]                    # top-middle 2x2 block
```

Boolean (mask) indexing

PYTHON

```python
x = np.array([-2, 5, -1, 8, 0])
mask = x > 0                  # array([False, True, False, True, False])
x[mask]                       # array([5, 8])   (copy)
x[x < 0] = 0                  # clamp negatives in place → [0 5 0 8 0]

# Combine conditions with & | ~ (NOT Python and/or), parenthesize!
x[(x > 1) & (x < 8)]
```

Fancy (integer-array) indexing

PYTHON

```python
a = np.array([10, 20, 30, 40, 50])
a[[0, 2, 4]]                  # array([10, 30, 50])  (copy)
a[[0, 2, 4]] = -1             # scatter-assign

M = np.arange(12).reshape(3, 4)
M[[0, 2], [1, 3]]             # array([1, 11])  pairs (0,1) and (2,3)
```

Views alias memory

Because a slice is a view, writing to it mutates the original: `b = a[2:5]; b[0] = 99` changes `a[2]`. If you need independence, call `.copy()`.

Reshaping and Combining

PYTHON

```python
a = np.arange(12)
a.reshape(3, 4)              # (3,4) view, same data
a.reshape(3, -1)             # -1 means "infer this axis" → (3,4)
a.reshape(2, 6).T            # transpose → (6,2) view
a.ravel()                    # flatten to 1-D (view if possible)
a.flatten()                  # flatten to 1-D (always a copy)

x = np.array([1, 2, 3])
x[:, np.newaxis]             # (3,1) column; np.newaxis adds an axis
x[np.newaxis, :]             # (1,3) row

# Stacking / joining
np.concatenate([a[:3], a[3:6]])        # along existing axis
np.vstack([[1, 2], [3, 4]])            # stack rows → (2,2)
np.hstack([[1, 2], [3, 4]])            # → [1 2 3 4]
np.stack([np.zeros(3), np.ones(3)])    # new axis → (2,3)
```

Aggregation and Axis Semantics

Reductions collapse one or more axes. The key insight: `axis=k` is the axis that *disappears*.

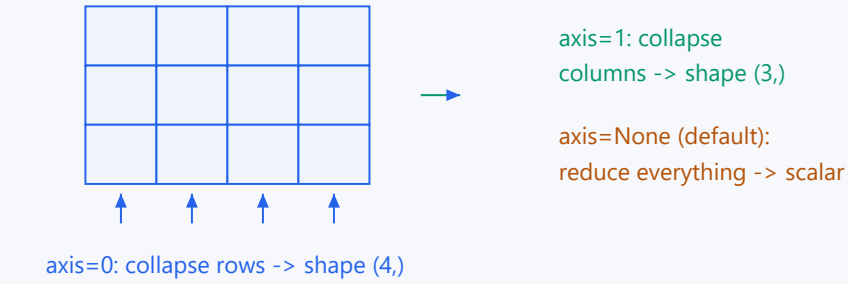


Fig 4. The axis you pass is the one that is summed away.

PYTHON

```python
M = np.arange(12).reshape(3, 4)
M.sum()                      # 66  (all elements)
M.sum(axis=0)                # [12 15 18 21]  column sums, shape (4,)
M.sum(axis=1)                # [ 6 22 38]     row sums, shape (3,)
M.mean(axis=0)               # column means
M.max(axis=1)                # per-row max
M.sum(axis=0, keepdims=True) # shape (1,4) — keeps dim for broadcasting

# argmin/argmax give the index of the extreme value
np.argmax(M, axis=1)         # [3 3 3]
np.cumsum(M, axis=1)         # running totals along each row
```

Function	Meaning
`sum, prod`	Total / product
`mean, median, std, var`	Descriptive statistics
`min, max, ptp`	Extremes; peak-to-peak range
`argmin, argmax`	Index of extreme
`cumsum, cumprod`	Running aggregates
`all, any`	Boolean reductions
`nanmean, nansum`	NaN-aware variants

Linear Algebra, Random, and More

Linear algebra (`np.linalg`)

PYTHON

```
A = np.array([[2., 1.], [1., 3.]])
b = np.array([1., 2.])

A @ b                       # matrix-vector product (@ is matmul)
A.T                         # transpose
np.linalg.inv(A)            # inverse (prefer solve over inv)
x = np.linalg.solve(A, b)   # solve A x = b  (stable, faster than inv)
np.linalg.det(A)            # determinant
vals, vecs = np.linalg.eig(A)       # eigen-decomposition
U, s, Vt = np.linalg.svd(A)         # singular value decomposition
np.linalg.norm(b)           # Euclidean norm
```

✦ **solve, don't invert**

To solve `Ax=b`, use `np.linalg.solve(A, b)`, not `inv(A) @ b`. It is faster and numerically more stable because it avoids explicitly forming the inverse.

Random (modern Generator API)

PYTHON

```
# NEW style (recommended since NumPy 1.17): explicit Generator + seed
rng = np.random.default_rng(seed=42)
rng.random((2, 3))            # uniform [0,1)
rng.integers(0, 10, size=5)   # random ints in [0,10)
rng.normal(loc=0, scale=1, size=1000)   # Gaussian samples
rng.choice([1, 2, 3], size=4, replace=True)
rng.shuffle(my_array)         # in-place permutation

# OLD global API (legacy, still works but avoid in new code):
# np.random.seed(0); np.random.rand(3)
```

i **Reproducibility**

Prefer `np.random.default_rng(seed)` over the legacy global `np.random.seed`. The Generator is faster, uses a better algorithm (PCG64), and avoids hidden global state that makes tests flaky.

Worked Mini-Example: K-Means Assignment Step

Assign 1000 points to the nearest of 4 centroids — fully vectorized, no Python loop over points. This showcases broadcasting + aggregation together.

PYTHON

```python
import numpy as np
rng = np.random.default_rng(0)

points    = rng.random((1000, 2))     # (N, D) = (1000, 2)
centroids = rng.random((4, 2))        # (K, D) = (4, 2)

# Broadcast points (N,1,D) against centroids (1,K,D) → diffs (N,K,D)
diffs = points[:, np.newaxis, :] - centroids[np.newaxis, :, :]
dists = np.sqrt((diffs ** 2).sum(axis=2))   # (N, K) distances

labels = np.argmin(dists, axis=1)            # (N,) nearest centroid index
print(labels[:10])                           # e.g. [2 0 3 1 ...]

# Recompute each centroid as the mean of its assigned points
new_centroids = np.stack([points[labels == k].mean(axis=0)
                          for k in range(4)])
```

The distance matrix for 1000x4 points is computed in a single broadcasted expression — the equivalent nested Python loop would be dozens of times slower and far harder to read.

Cheat-Sheet Reference

Task	Code
Create range	`np.arange(n)` / `np.linspace(a,b,n)`
Shape / dims / size	`a.shape` / `a.ndim` / `a.size`
Change shape	`a.reshape(r,c)` , `a.ravel()` , `a.T`
Add axis	`a[:, None]` or `np.newaxis`
Element math	`a + b` , `np.sqrt(a)` , `a @ b`
Reduce	`a.sum(axis=0)` , `a.mean(axis=1)`
Boolean filter	`a[a > 0]` , `np.where(cond, x, y)`
Deep copy	`a.copy()`
Stack	`np.vstack` , `np.hstack` , `np.concatenate`
Solve linear system	`np.linalg.solve(A, b)`
Random	`np.random.default_rng(seed)`
Save / load	`np.save('f.npy', a)` / `np.load('f.npy')`

NumPy array vs Python list

Aspect	Python list	NumPy ndarray
Element type	Mixed, boxed objects	Single fixed dtype
Memory	Pointers, scattered	Contiguous buffer
Element math	Manual loop	Vectorized ufuncs
Speed on numerics	Slow (interpreter)	Fast (C + SIMD)
N-D support	Nested lists only	Native, with strides
Resizable	Yes (append cheap)	Fixed size (append copies)

Best Practices and Common Pitfalls

✓ **Do**

Vectorize; preallocate with `zeros/empty` and fill via slices; pass `axis=` explicitly; use `default_rng` ; choose `float32` when memory-bound and `float64` when precision-bound; use `@` for matmul.

✦ **Tip**

Use `np.where(cond, a, b)` for element-wise conditionals and `np.clip(a, lo, hi)` to bound values — both avoid loops. `keepdims=True` keeps a reduction broadcast-compatible with the original.

⚠ **Pitfall: view vs copy**

Slices alias the parent. Mutating a slice mutates the original. When in doubt, `.copy()`. Fancy/boolean indexing already copies.

⚠ **Pitfall: integer overflow**

`int8/uint8` wrap silently. Summing a large `uint8` image can overflow — cast up with `.astype(np.int64)` first or use `dtype=` in the reduction.

✗ **Pitfall: == with floats and NaN**

`0.1 + 0.2 == 0.3` is `False` ; use `np.isclose` / `np.allclose` . And `np.nan == np.nan` is `False` — test with `np.isnan` .

✗ **Pitfall: and/or on arrays**

Python `and` / `or` raise "ambiguous truth value" on arrays. Use element-wise `&` , `|` , `~` and parenthesize each comparison.

Interview Questions & Answers

Why is a NumPy array faster than a Python list for numeric work?

It stores raw, fixed-width values in one contiguous buffer instead of pointers to boxed objects. Operations run in a single compiled C loop (with cache locality and SIMD) rather than per-element interpreter dispatch, and there is no per-element boxing/unboxing. This commonly yields 10–100x speedups.

What is broadcasting and what are its rules?

Broadcasting lets operations combine arrays of different shapes without copying. Compare shapes right-to-left: dimensions are compatible if equal or if one is 1 (or missing). Size-1 axes are virtually stretched. Example: `(3,1) + (1,4) → (3,4)`.

What does `axis=0` mean in `a.sum(axis=0)`?

It reduces along axis 0 — that axis collapses. For a 2-D array, `axis=0` sums down the rows producing one value per column; `axis=1` sums across columns producing one value per row. The axis you name is the one that disappears.

When does indexing return a view versus a copy?

Basic slicing (`a[1:5]`, `a[:, 0]`) returns a view sharing memory. Fancy indexing (integer arrays) and boolean-mask indexing return a copy. Mutating a view mutates the parent; copies are independent. Check with `arr.base`.

What is a dtype, and why does it matter?

The dtype is the fixed element type (e.g. `float64`, `int32`). It determines memory per element, numeric range/precision, and overflow behavior. All elements share it, which is what makes the buffer homogeneous and fast. Mixing dtypes triggers type promotion to a common type.

What does `reshape(-1)` do, and what is the difference between `ravel` and `flatten`?

`-1` tells NumPy to infer that axis's length from the total size. `ravel()` returns a flattened 1-D view when memory layout allows (cheap); `flatten()` always returns a fresh copy.

How do you solve a linear system `Ax = b`, and why not use the inverse?

Use `np.linalg.solve(A, b)`. It performs an LU factorization and back-substitution, which is faster and numerically more stable than computing `np.linalg.inv(A) @ b`, since forming an explicit inverse amplifies rounding error.

Why can't you use Python's `and` / `or` to combine array conditions?

`and` / `or` try to evaluate the truth value of the whole array, which is ambiguous and raises. Use element-wise bitwise operators `&`, `|`, `~`, and wrap each comparison in parentheses because of operator precedence: `(x > 0) & (x < 10)`.

How would you standardize each column of a matrix without a loop?

`(X - X.mean(axis=0)) / X.std(axis=0)`. The column statistics have shape `(n_cols,)` and broadcast against the `(n_rows, n_cols)` matrix, subtracting/dividing element-wise with no explicit iteration.

What is the difference between `np.array` and `np.asarray`?

`np.array(x)` always makes a copy by default. `np.asarray(x)` avoids copying if `x` is already an ndarray of the right dtype — it returns the input unchanged. Use `asarray` at API boundaries to accept list-or-array input cheaply.

How do you handle missing values (NaN) in aggregations?

Regular reductions propagate NaN (`mean` of data containing NaN is NaN). Use NaN-aware variants like `np.nanmean`, `np.nansum`, `np.nanmax`. Detect NaN with `np.isnan` (never `== np.nan`), and build masks to drop or impute them.

What is a ufunc?

A universal function is a vectorized function that operates element-wise over arrays with broadcasting, type promotion, and optional `out=` / `where=` support — e.g. `np.add`, `np.sin`, `np.maximum`. Reductions like `.sum()` are implemented as ufunc reductions (`np.add.reduce`).

Further Learning

- Official **NumPy user guide** and the "NumPy: the absolute basics for beginners" tutorial.
- Learn **pandas** next — it wraps NumPy arrays with labeled axes for tabular data.
- Study **strides and** `np.lib.stride_tricks.sliding_window_view` for zero-copy windowing.
- Explore **structured/record arrays** and **masked arrays** for heterogeneous and missing data.
- Move to **SciPy** (optimization, sparse, signal) and **Numba**/**CuPy** for JIT and GPU acceleration of array code.
- Benchmark your own loop-vs-vectorized rewrites with `%timeit` to build intuition.